

Corso di Laurea Magistrale in Ingegneria e Scienze Informatiche

Modernizing a Spark-Based ETL Platform Using Transactional Lakehouse Technologies on AWS

Relatore

Prof. Boschetti Marco Antonio

Candidato

Tommaso Brini

Correlatori

Dott. CoSupervisor 1

Dott. CoSupervisor 2

Azienda

PROMETEIA S.P.A

Abstract

Questa tesi analizza la modernizzazione di una piattaforma ETL basata su Apache Spark e AWS Glue, precedentemente vincolata a un'architettura Data Lake tradizionale sul formato Parquet. L'obiettivo centrale del lavoro è l'introduzione del paradigma Transactional Lakehouse tramite l'adozione di Apache Iceberg. L'introduzione di questo table format ha permesso di abilitare proprietà ACID, funzionalità di time travel e una gestione avanzata dei metadati.

Nel corso del tirocinio è stato progettato e implementato un prototipo in grado di gestire pipeline complesse (INSERT, UPDATE, DELETE e MERGE) su dataset di grandi dimensioni, includendo una strategia automatizzata per la sincronizzazione del catalogo e dei metadati.

La soluzione proposta viene quindi confrontata con l'architettura esistente per valutarne gli effetti in termini di prestazioni, scalabilità e robustezza operativa, evidenziando vantaggi e limiti delle tecnologie lakehouse nel contesto di pipeline ETL in ambiente cloud.

Contents

Abstract	iii
1 Introduzione	1
2 Stato dell'arte e tecnologie	5
2.1 Evoluzioni delle Architetture Dati	5
3 Analisi As-is: Architettura Esistente	7
4 Progettazione e Implementazione	9
5 Test e risultati	11
6 Conclusioni e sviluppi futuri	13
Bibliography	15

CONTENTS

Chapter 1

Introduzione

Il presente lavoro di tesi è stato sviluppato nell’ambito di un tirocinio svolto presso Prometeia S.p.A., azienda attiva nella consulenza e nello sviluppo di soluzioni software per il settore finanziario.

Negli ultimi anni la crescita esponenziale dei dati generati da applicazioni distribuite, sistemi transazionali e servizi cloud ha reso necessaria l’evoluzione delle architetture aziendali per la gestione e l’analisi dei dati. In questo contesto, i processi di **Extract, Transform, Load (ETL)** hanno assunto un ruolo centrale nelle pipeline di data engineering, richiedendo soluzioni sempre più scalabili, affidabili e flessibili.

La diffusione dei paradigmi Big Data ha favorito l’adozione dei Data Lake, architetture basate su storage distribuito in grado di memorizzare e gestire grandi quantità di dati eterogenei a costi ridotti. Formati colonnari come Apache Parquet hanno quindi rappresentato uno standard de facto per l’elaborazione distribuita tramite framework come Apache Spark. Nel modello Lakehouse, i Data Lake moderni si basano infatti su storage cloud economico e formati aperti per supportare workload analitici e di machine learning. [AGX⁺21] [BGK21]

Nonostante i vantaggi in termini di costo e scalabilità, l’approccio tradizionale basato su file Parquet presenta alcune limitazioni significative. In particolare, i Data Lake classici non forniscono nativamente proprietà transazionali ACID, una gestione evoluta dei metadati o un supporto efficiente per operazioni di aggiornamento incrementale. Operazioni come UPDATE, DELETE e MERGE richiedono

quindi meccanismi applicativi complessi, spesso basati sulla riscrittura di intere partizioni o dataset, con un aumento della complessità delle pipeline ETL e del rischio di inconsistenze nei dati.

Per superare tali criticità si è diffuso il paradigma del Data Lakehouse, che combina la flessibilità dei Data Lake con caratteristiche tipiche dei Data Warehouse, introducendo transazioni, versionamento dei dati, schema evolution e una gestione avanzata dei metadati. [Sah24]

Tra le tecnologie emergenti in questo ambito, Apache Iceberg si è affermato come uno dei principali table format open-source per la gestione di dataset analitici su larga scala. Iceberg introduce un layer di metadata management che separa la struttura logica della tabella dalla disposizione fisica dei file su storage object-based, consentendo la gestione di snapshot consistenti, time travel e operazioni incrementali transazionali. [Apa25a]

A differenza di un approccio puramente file-based, Iceberg mantiene lo stato della tabella attraverso metadata versionati e manifest file, consentendo aggiornamenti atomici mediante commit espliciti. [Apa25b]

Questa tesi si inserisce nel contesto della modernizzazione di una piattaforma ETL aziendale basata su Apache Spark e servizi AWS. L'architettura esistente utilizza file Parquet memorizzati su Amazon S3 ed elaborati tramite AWS Glue, presentando criticità legate alla gestione degli aggiornamenti incrementali, alla sincronizzazione del catalogo e alla garanzia di idempotenza dei processi batch.

L'obiettivo del lavoro è progettare e valutare una nuova architettura basata sul paradigma Transactional Lakehouse attraverso l'introduzione di Apache Iceberg. In particolare, il lavoro analizza l'integrazione del table format con Apache Spark e l'ecosistema AWS, includendo aspetti relativi alla gestione dei metadati, all'esecuzione di operazioni MERGE transazionali e ai processi di maintenance delle tabelle. Durante il progetto sono inoltre state esplorate le funzionalità offerte da **Amazon S3 Tables** [Ama25a], soluzione managed introdotta da AWS per la gestione di tabelle Iceberg.

Il contributo principale della tesi consiste nella progettazione e implementazione di un prototipo in grado di eseguire operazioni incrementali affidabili e idempotenti su dataset di grandi dimensioni, confrontando la nuova soluzione con l'architettura preesistente in termini di prestazioni, scalabilità e robustezza.

L'analisi sperimentale evidenzia inoltre i trade-off introdotti dall'adozione del paradigma lakehouse, valutando vantaggi e limiti sia di un'implementazione Iceberg tradizionale sia dell'approccio gestito offerto da S3 Tables.

Nei prossimi capitoli, la tesi introdurrà lo stato dell'arte relativo alle architetture Data Lake e Lakehouse, descrivendo le principali tecnologie utilizzate (Capitolo 2). Successivamente, verrà presentata un'analisi dettagliata dell'architettura esistente e delle criticità riscontrate (Capitolo 3), seguita dalla progettazione e implementazione della nuova soluzione basata su Apache Iceberg (Capitolo 4). Infine, i risultati sperimentali e i benchmark effettuati saranno discussi nel Capitolo 5, raccogliendo conclusioni e possibili sviluppi futuri nel Capitolo 6.

Chapter 2

Stato dell'arte e tecnologie

- L'evoluzione delle Architetture Dati: Dal Data Warehouse al Data Lake, fino al paradigma Data Lakehouse.
- Apache Spark: Architettura e ruolo nell'elaborazione distribuita.
- I Table Format Moderni: Analisi comparativa tra Parquet "puro" e Apache Iceberg.
- Focus su Iceberg: Snapshot, Schema Evolution, Partition Evolution e Time Travel.
- Ecosistema AWS: Panoramica dei servizi utilizzati (S3, Glue Catalog, AWS Glue Jobs, Athena, S3 Tables).

2.1 Evoluzioni delle Architetture Dati

Chapter 3

Analisi As-is: Architettura Esistente

- Pipeline ETL esistente: Descrizione del flusso dati basato su Spark e file Parquet.
- Pain Points: Analisi dettagliata delle inefficienze riscontrate, i 6 punti critici del documento di formazione.

Chapter 4

Progettazione e Implementazione

- Design della soluzione (To-Be): Come cambia l'architettura logica.
- Configurazione dello Stack: Integrazione di Iceberg con Spark su AWS.
- Ottimizzazione: Implementazione di processi di Maintenance (Compaction dei file, rimozione dei vecchi snapshot/orphan files).

Chapter 5

Test e risultati

- Benchmark di Performance: Confronto tra il vecchio sistema e il nuovo (tempi di esecuzione del batch).
- Affidabilità: Dimostrazione del supporto alle transazioni ACID (es. cosa succede se un job fallisce a metà).
- Costi: Impatto dell'ottimizzazione dei file (Compaction) sui costi di storage.
- Risposta ai 6 punti critici

Chapter 6

Conclusioni e sviluppi futuri

- Risultati raggiunti: Sintesi dei benefici ottenuti dall'azienda.
- Limiti riscontrati: Eventuali criticità del framework o dei servizi AWS durante lo sviluppo.
- Evoluzioni: aggiornamento parziale sui record compositi

Bibliography

- [AGX⁺21] Michael Armbrust, Ali Ghodsi, Reynold Xin, Matei Zaharia, et al. Lakehouse: a new generation of open platforms that unify data warehousing and advanced analytics. *Proceedings of CIDR*, 8(1):28, 2021.
- [Ama25a] Amazon Web Services. Amazon s3 tables documentation. <https://docs.aws.amazon.com/AmazonS3/latest/userguide/s3-tables.html>, 2025.
- [Ama25b] Amazon Web Services. Aws glue documentation, 2025. <https://docs.aws.amazon.com/glue/>.
- [Apa25a] Apache Software Foundation. Apache iceberg documentation, 2025. <https://iceberg.apache.org>.
- [Apa25b] Apache Software Foundation. Apache iceberg specification. <https://apache.github.io/iceberg/spec/>, 2025.
- [Apa25c] Apache Software Foundation. Apache parquet documentation. <https://parquet.apache.org/docs/>, 2025.
- [Apa25d] Apache Software Foundation. Apache spark documentation. <https://spark.apache.org/docs/latest/>, 2025.
- [BGK21] Edmon Begoli, Ian Goethert, and Kathryn Knight. A lakehouse architecture for the management and analysis of heterogeneous data for biomedical research and mega-biobanks. In *2021 IEEE International Conference on Big Data (Big Data)*, pages 4643–4651. IEEE, 2021.

BIBLIOGRAPHY

- [Sah24] Dipankar Saha. Disruptor in data engineering-comprehensive review of apache iceberg. *SSRN Electronic Journal*, 2024.

Acknowledgements

Optional. Max 1 page.